MULTI-AGENT COUNCIL

n8n workflow: one master agent, five worker agents, Telegram and localhost interface

Purpose	Combine 2 ChatGPT, 2 Claude and 2 Gemini API accounts into one system. A master agent plans the workflow for each prompt, worker agents execute it in parallel, the master combines the results, a reviewer checks them, and the answer returns to Telegram and a localhost page.
Runs on	Docker Compose: n8n (editor 127.0.0.1:5678) and a small Node web server (127.0.0.1:8080). Everything is bound to this machine only.
Deliverables	multi-agent-orchestrator.json (13-node workflow), web/ (localhost console), docker-compose.yml, .env.example, README, this document.
Verified	18 offline tests of the Code-node logic against mock APIs, plus an end-to-end run in a real n8n 2.35.7 instance (import, activate, Telegram path, web path, failover, bad-secret rejection).

1 BEFORE ANYTHING ELSE: WHAT "FREE KEYS" MEANS

The chat apps and the developer APIs are separate products with separate billing. n8n talks to the API, so each of the six slots needs an **API key**, not a ChatGPT, Claude or Gemini app login.

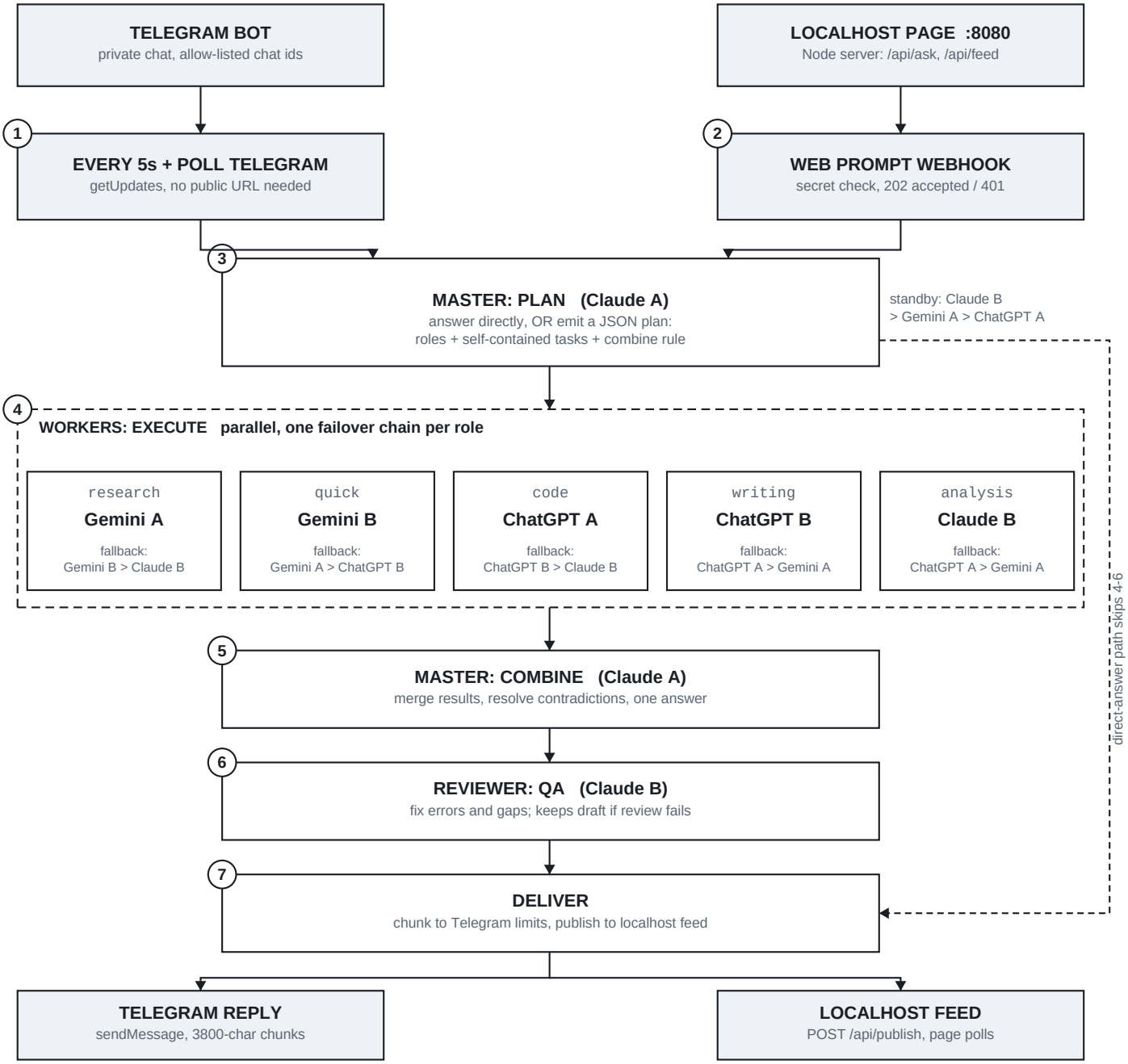
Vendor	Chat-app account gives you	API key	Free route
Google Gemini	Gemini app	Created in Google AI Studio	**Yes.** Free tier covers Flash and Flash-Lite class models (gemini-3.5-flash, gemini-3.1-flash-lite). The Pro model is listed with no free tier. Limits are per project, so two keys in two separate projects give separate quotas.
OpenAI	ChatGPT Free/Plus/Go	Separate platform billing	**No general free tier.** Only paid Pro and Business plans receive monthly API credits; Free, Plus and Go are not eligible. Expect to add prepaid credit.
Anthropic	Claude.ai plan	Separate Console billing	**No free tier assumed.** Claude API usage is billed separately from a Claude plan. Expect to add prepaid credit.

Consequence for the design. Every agent is optional: a slot without a key is skipped and its work falls through to the next agent in the chain. Three operating modes are supported with no code change:

Mode	Keys present	Behaviour
Full council	all six	Strategy exactly as drawn in section 3.
Free-only	Gemini A + B	Set MASTER_CHAIN=gemini_a,gemini_b. Gemini plans, works, combines and reviews (set REVIEWER_AGENT=gemini_b). Tested.
Partial	any subset	Missing agents are skipped; roles fall back along their chains.

Not supported by design: using a chat subscription through its web session or cookies. That is not an API, breaks without notice and is against the vendors' terms.

2 SYSTEM ARCHITECTURE



PROJECT	Multi-Agent Council	DRAWING	A-01 Orchestration flow	REV	A
SCALE	none (schematic)	DATE	2026-11-11	SHEET	2 / 6

3 MASTER AND WORKER STRATEGY

Decision. Claude A is the master. Claude B is its hot standby and doubles as the analysis worker and reviewer. The other four agents are specialists grouped by vendor so that one vendor's outage or rate limit never removes a whole role.

Why the master is a Claude agent (design judgement, not a benchmark result):

- The master must return a strict JSON plan and then write one coherent final answer from several partial outputs. Instruction-following and long-form synthesis matter more than raw speed here.
- The master makes only two calls per prompt (plan, combine), so it is the cheapest place to spend the strongest model; workers make most of the calls and run on lighter models.
- Claude B is a second, independent account of the same vendor, so master failover keeps identical behaviour and prompt format.
- If you only have free keys, the chain moves to Gemini (section 1). The role split still holds because roles are configuration, not code.

Role assignment and failover chains

Role	Used for	Primary	Failover order
master	plan, combine	Claude A	Claude B, Gemini A, ChatGPT A, Gemini B, ChatGPT B
research	broad knowledge, long reading, gathering and summarising	Gemini A	Gemini B, Claude B, ChatGPT A
quick	extraction, translation, formatting, short summaries	Gemini B	Gemini A, ChatGPT B, Claude B
code	programming, debugging, maths, structured data	ChatGPT A	ChatGPT B, Claude B, Gemini A
writing	drafting, rewriting, tone, alternative angles	ChatGPT B	ChatGPT A, Gemini A, Claude B
analysis	reasoning, critique, trade-offs, risk	Claude B	ChatGPT A, Gemini A, ChatGPT B
reviewer	final quality gate	Claude B	Gemini A, ChatGPT A, Claude A

After its listed chain a worker task may still use any remaining agent as a last resort. Every chain can be overridden in .env (MASTER_CHAIN, ROLE_RESEARCH, ROLE_QUICK, ROLE_CODE, ROLE_WRITING, ROLE_ANALYSIS, REVIEWER_AGENT).

How the master "creates the workflow"

For each prompt the master returns one of two JSON shapes. The worker stage is then built from the plan at run time, so the workflow is different for every prompt:

```
{"mode":"direct","answer":"..."}
{"mode":"delegate","summary":"...","combine":"how to merge",
 "tasks":[{"id":"t1","role":"research|quick|code|writing|analysis",
  "title":"...","instruction":"self-contained instruction"}]}
```

Limits applied by the workflow: 1 to 5 tasks, unknown roles become analysis, tasks cannot see each other (so instructions must be self-contained), malformed JSON falls back to a single analysis task, and a plain-text reply is treated as a direct answer.

4 WORKFLOW NODE REFERENCE

#	Node	Type	Job
1	Every 5s -> Poll Telegram	Schedule trigger + Code	Calls getUpdates (no public URL, so it works on localhost). Ignores chats not in the allow-list; in setup mode replies with the sender's chat id. Handles /start, /help, /agents. Sends an acknowledgement, then emits one job per message.
2	Web Prompt -> Validate -> Valid? -> Respond 202 / 401	Webhook, Code, IF, Respond	POST /webhook/agents-prompt with header x-webhook-secret. Wrong secret or empty prompt gets 401; otherwise 202 returns immediately and the run continues in the background.
3	Master: Plan	Code	Master chain call. Direct answer, or validated plan (section 3).
4	Workers: Execute	Code	Runs all tasks with Promise.all. Each task walks its role chain until one agent answers.
5	Master: Combine	Code	Merges worker output into one answer. If every master candidate fails, returns the raw worker outputs instead of an error.
6	Reviewer: QA	Code	Skipped for direct answers, short drafts and degraded runs, or when REVIEW_ENABLED=false. Keeps the draft if the reviewer fails or returns under 40% of its length.
7	Deliver: Telegram + Localhost	Code	POSTs the result to the page's /api/publish and sends the Telegram reply (prompts typed on the page are mirrored to the first allow-listed chat unless MIRROR_WEB_TO_TELEGRAM=false).

Why Code nodes with one shared gateway instead of one node per agent

The plan decides at run time which agents run and in which order they fail over, which is awkward to wire statically. One gateway library (Anthropic Messages, OpenAI Responses, Gemini generateContent adapters, per-agent cooldowns, key scrubbing) is injected into each Code node from workflows/src/lib.js by scripts/build_workflow.js. Keys come from environment variables, so no credentials are stored in the workflow file and nothing needs relinking after import.

Localhost console (web/)

- Zero-dependency Node server. /api/ask creates a job id, forwards the prompt to the n8n webhook with the secret, and returns at once. The browser polls /api/feed.
- n8n calls /api/publish (secret-protected) when a job finishes, from either channel, so Telegram conversations also appear on the page.
- The feed is kept in memory and mirrored to web/data/feed.json (last 200 items). A job with no reply after 10 minutes is shown as failed with a hint.
- Answers render through an escape-first mini markdown renderer; no external scripts or fonts are loaded.

5 REQUEST LIFECYCLE

Step	Telegram prompt	Localhost prompt
1	Poller picks up the message within 5 s and replies "Master is planning".	Browser POSTs /api/ask; server stores a pending item and calls the n8n webhook.
2	Job {jobId, source:telegram, chatId, prompt}.	n8n validates the secret and answers 202; job {jobId, source:web, prompt}.
3	Master plans; workers run in parallel; master combines; reviewer checks.	Same pipeline.
4	Reply sent to the originating chat in chunks of at most 3800 characters, with a footer naming the agents used.	Answer appears in the page; copy is sent to the first allow-listed Telegram chat.
5	Result also published to the page feed.	Pending bubble becomes the answer, with chips showing master, workers (dashed = a fallback took over), reviewer and time.

6 FAILURE HANDLING

Event	Behaviour
HTTP 429 from an agent	Next agent in the chain answers; the limited agent is skipped for 60 s.
HTTP 5xx or network error	Next agent answers; 30 s cooldown (60 s for network errors).
HTTP 401 or 403 (bad key)	Skipped for 15 minutes so a bad key is not retried on every task.
HTTP 404 (wrong model name)	Skipped for 10 minutes. Fix the model in .env.
All agents in a chain cooling down	They are tried once anyway as a last resort.
Master plan unparseable	Single analysis task runs; raw JSON is never shown to the user.
Some workers fail	Combine proceeds with the successful ones and notes the gaps.
All combiners fail	Raw worker outputs are returned (flagged as unmerged).
Nothing can answer	A clear message reaches both channels, for example "no agent has an API key configured".
Provider error text contains a key	API-key and bot-token patterns are redacted before any message is sent.

Cooldowns live in workflow static data, which n8n persists only for active (production) runs. Cost control: a prompt costs one plan call, up to five worker calls, one combine call and one review call; set REVIEW_ENABLED=false or lower WORKER_MAX_TOKENS to reduce it.

7 SECURITY

- Telegram: only chat ids in TELEGRAM_ALLOWED_CHAT_IDS can spend your keys; other senders get no reply. While the list is empty the bot only reveals the sender's own chat id.
- Webhook and publish endpoints require WEBHOOK_SECRET (constant-time compare on the publish side). The page server refuses to start without it.
- n8n and the page publish ports on 127.0.0.1 only. Do not expose them to the internet without authentication in front.
- Keys live only in .env (never in the workflow JSON). N8N_BLOCK_ENV_ACCESS_IN_NODE=false is required for Code nodes to read them; that setting lets any workflow author on this n8n read all environment variables, which is acceptable for a single-user local install.
- Executions that succeed are not stored (the poller would otherwise create about 17,000 rows a day); errors are kept and pruned after 48 hours. Switch saveDataSuccessExecution on in workflow settings when debugging.

8 SETUP AND VERIFICATION

Step	Action
1	cp .env.example .env. Set WEBHOOK_SECRET (openssl rand -hex 24) and whichever API keys you have.
2	Telegram: create a bot with @BotFather, put the token in TELEGRAM_BOT_TOKEN.
3	docker compose up -d. Open http://localhost:5678 and create the n8n owner account.
4	docker compose exec n8n n8n import:workflow --input=/import/multi-agent-orchestrator.json, then restart n8n (docker compose restart n8n). Alternatively use Workflows > Import from file.
5	Open the workflow and switch it to Active (production webhooks and the 5 s poller only run when active).
6	Message your bot. It replies with your chat id; put it in TELEGRAM_ALLOWED_CHAT_IDS and run docker compose up -d again.
7	Send /agents in Telegram: every agent with a key should show as ready. Then send a real prompt, or open http://localhost:8080.

Verification already performed

Check	Result
Workflow JSON: 13 nodes, all connections resolve, every Code node parses	pass
18 offline scenarios (allow-list, setup mode, direct answer, delegate path, worker and master failover, whole-vendor outage, key subset, no keys, key redaction, malformed plan, degraded combine, secret validation, chunking, web proxy, mirror toggle)	18 / 18 pass
Real n8n 2.35.7: CLI import, publish, activation, webhook 401 on bad secret	pass
Real n8n: Telegram prompt -> plan -> 3 parallel workers -> combine -> review -> Telegram reply and page feed	pass
Real n8n: page prompt -> same pipeline -> feed and Telegram mirror; gemini_a forced to 429 -> Gemini B took over	pass

Not verified: calls to the real OpenAI, Anthropic and Gemini endpoints (no keys were available here, so the vendor adapters follow the published request formats and were tested against mocks), the Docker Compose stack itself, and a real Telegram bot. The first live run is the real test of those three; /agents and a single "hello" prompt are the quickest checks. If an OpenAI-compatible endpoint rejects /v1/responses, set OPENAI_API_MODE=chat.

9 EXTENDING

- Add a role: add an entry to ROLES and ROLE_INFO in workflows/src/lib.js, then run node scripts/build_workflow.js and re-import. The master learns the new role automatically from the role list in its prompt.
- Point any slot at another provider: set OPENAI_BASE_URL (or the Anthropic/Gemini equivalent) to an OpenAI-compatible gateway and set the model name.
- Dependent tasks (task B needs the output of task A): not in this version; tasks run in a single parallel wave. The combine step is where results meet.
- Rebuild and test: node scripts/build_workflow.js && node tests/run_nodes.js.

10 SOURCES CHECKED

- Gemini API pricing and free-tier model list: https://ai.google.dev/gemini-api/docs/pricing
- Gemini API rate limits (per project, free usage tier): https://ai.google.dev/gemini-api/docs/rate-limits
- OpenAI: API billed separately from ChatGPT; credits only for eligible paid plans: https://developers.openai.com/api/docs/guides/chatgpt-api-credits
- OpenAI model IDs (gpt-6 family): https://developers.openai.com/api/docs/models
- Anthropic: "Why is Claude API usage billed separately from my paid Claude plan" (support.claude.com article 9876003). The page itself could not be fetched here; the title was read from search results.
- Model names and free-tier details change often; confirm them against the vendor pages before relying on them.